

ASSIGNMENT – 7.1

NAME : G.RISHITHA

HT.NO : 2303A52149

BATCH : 35

Task Description -1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., print "Hello"). Use AI to detect and fix the syntax error.

```
# Bug: Missing parentheses in print statement
def greet():
print "Hello, AI Debugging Lab!"
greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.

- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1:

- Corrected code with proper syntax and AI explanation.

```
Task_1_Syntax_Error.py X Task_1_REPORT.txt
Task_1_Syntax_Error.py > ...
1  # CORRECTED CODE
2  # Bug Fixed: Added parentheses in print statement
3  def greet():
4      print("Hello, AI Debugging Lab!")
5
6  # Assertion test cases to verify the fix works
7  def test_greet():
8      """Test cases to verify the corrected greet function"""
9
10     # Test case 1: Verify function executes without error
11     try:
12         greet()
13         assert True, "Test 1 passed: greet() executed successfully"
14         print("✓ Test 1 passed: greet() executed successfully")
15     except Exception as e:
16         assert False, f"Test 1 failed: {e}"
17
18     # Test case 2: Verify function returns None (as expected for print function)
19     result = greet()
20     assert result is None, "Test 2 failed: greet() should return None"
21     print("✓ Test 2 passed: greet() returns None as expected")
22
23     # Test case 3: Verify the greeting message format
24     import io
25     import sys
26
27     # Capture print output
28     captured_output = io.StringIO()
29     sys.stdout = captured_output
```

```

Task_1_Syntax_Error.py > ...
7  def test_greet():
16      assert False, f"Test 1 failed: {e}"
17
18      # Test case 2: Verify function returns None (as expected for print function)
19      result = greet()
20      assert result is None, "Test 2 failed: greet() should return None"
21      print("✓ Test 2 passed: greet() returns None as expected")
22
23      # Test case 3: Verify the greeting message format
24      import io
25      import sys
26
27      # Capture print output
28      captured_output = io.StringIO()
29      sys.stdout = captured_output
30      greet()
31      sys.stdout = sys.__stdout__
32
33      output = captured_output.getvalue()
34      assert "Hello, AI Debugging Lab!" in output, "Test 3 failed: Output doesn't contain expected mess
35      print("✓ Test 3 passed: Output contains 'Hello, AI Debugging Lab!'")
36
37      # Run the corrected function
38      greet()
39
40      # Run all test cases
41      print("\n--- Running Test Cases ---")
42      test_greet()

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode> & C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/
● Hello, AI Debugging Lab!

--- Running Test Cases ---
Hello, AI Debugging Lab!
✓Test 1 passed: greet() executed successfully
Hello, AI Debugging Lab!
✓Test 2 passed: greet() returns None as expected
✓Test 3 passed: Output contains 'Hello, AI Debugging Lab!'
○ PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode>

```

Task Description - 2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):
```

```
    if n = 10:
```

```
        return "Ten"
```

```
    else:
```

```
        return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test execution.

Task_2_If_Condition_Bug.py > ...

```
1  # CORRECTED CODE
2  # Bug Fixed: Changed assignment (=) to comparison (==)
3  def check_number(n):
4      if n == 10:
5          return "Ten"
6      else:
7          return "Not Ten"
8
9  # =====
10 # EXPLANATION OF THE BUG:
11 # =====
12 # The original code used a single equals sign (=) in the if condition:
13 #     if n = 10:
14 #
15 # This is WRONG because:
16 # 1. The single = operator is for ASSIGNMENT, not comparison
17 # 2. It tries to assign the value 10 to variable n
18 # 3. You cannot use an assignment statement inside an if condition
19 # 4. This results in a SyntaxError: "Expected ':'"
20 #
21 # The correct syntax uses the double equals (==) operator for comparison:
22 #     if n == 10:
23 #
24 # This properly COMPARES whether n is equal to 10 and returns True/False
25 #
26 # KEY DIFFERENCE:
27 #     = (single) → Assignment operator (assigns a value to a variable)
28 #     == (double) → Comparison operator (compares two values for equality)
29 # =====
```

```

Task_2_If_Condition_Bug.py > ...
28 # == (double) → Comparison operator (compares two values for equality)
29 # =====
30
31 # =====
32 # TEST CASES
33 # =====
34
35 def test_check_number():
36     """Three comprehensive test cases for the check_number function"""
37
38     # Test Case 1: Input equals 10
39     result1 = check_number(10)
40     assert result1 == "Ten", f"Test 1 failed: Expected 'Ten', got '{result1}'"
41     print("✓ Test 1 PASSED: check_number(10) returns 'Ten'")
42
43     # Test Case 2: Input does not equal 10 (positive number)
44     result2 = check_number(5)
45     assert result2 == "Not Ten", f"Test 2 failed: Expected 'Not Ten', got '{result2}'"
46     print("✓ Test 2 PASSED: check_number(5) returns 'Not Ten'")
47
48     # Test Case 3: Input is negative or zero
49     result3 = check_number(-10)
50     assert result3 == "Not Ten", f"Test 3 failed: Expected 'Not Ten', got '{result3}'"
51     print("✓ Test 3 PASSED: check_number(-10) returns 'Not Ten'")
52
53     # Run the corrected function
54     print("--- Testing Corrected Function ---")
55     test_check_number()
56     print("\nAll tests passed successfully!")

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode> & C:\U
n\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant C
y"
• --- Testing Corrected Function ---
✓Test 1 PASSED: check_number(10) returns 'Ten'
✓Test 2 PASSED: check_number(5) returns 'Not Ten'
✓Test 3 PASSED: check_number(-10) returns 'Not Ten'

All tests passed successfully!
○ PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode>

```

Task Description -3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()  
print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

Task_3_File_Not_Found_Bug.py > ...

```
1  # CORRECTED CODE
2  # Bug Fixed: Added exception handling with try-except block
3  import os
4
5  def read_file(filename):
6      """
7      Safely reads and returns the content of a file.
8
9      Args:
10         filename (str): The path to the file to read
11
12      Returns:
13         str: The file content if successful, error message if file not found
14      """
15      try:
16         # Check if file exists before attempting to open
17         if not os.path.exists(filename):
18             return f"ERROR: File '{filename}' does not exist."
19
20         # Check if it's actually a file (not a directory)
21         if not os.path.isfile(filename):
22             return f"ERROR: '{filename}' is not a valid file."
23
24         # Try to open and read the file
25         with open(filename, 'r') as f:
26             content = f.read()
27             return f"SUCCESS: File content read ({len(content)} characters)"
28
29     except FileNotFoundError:
```

Task_3_File_Not_Found_Bug.py > ...

```
5  def read_file(filename):
6
29     except FileNotFoundError:
30         return f"ERROR: FileNotFoundError - The file '{filename}' was not found."
31     except PermissionError:
32         return f"ERROR: PermissionError - You do not have permission to read '{filename}'."
33     except IsADirectoryError:
34         return f"ERROR: IsADirectoryError - '{filename}' is a directory, not a file."
35     except IOError as e:
36         return f"ERROR: IOError - Could not read file: {e}"
37     except Exception as e:
38         return f"ERROR: Unexpected error - {type(e).__name__}: {e}"
39
40 # =====
41 # EXPLANATION OF THE BUG:
42 # =====
43 # Original Code Problem:
44 #     with open(filename, 'r') as f:
45 #         return f.read()
46 #
47 # Issues:
48 # 1. No error handling - if file doesn't exist, program CRASHES
49 # 2. FileNotFoundError is raised and not caught
50 # 3. User doesn't get a helpful message
51 # 4. No graceful degradation or recovery
52 #
53 # Solution - Exception Handling:
54 # 1. Try block: Contains code that might raise an exception
55 # 2. Except block: Catches specific exceptions and handles them gracefully
56 # 3. User-friendly messages: Instead of cryptic errors
```


Task_3_File_Not_Found_Bug.py > ...

```
54 # 1. Try block: Contains code that might raise an exception
55 # 2. Except block: Catches specific exceptions and handles them gracefully
56 # 3. User-friendly messages: Instead of cryptic errors
57 # 4. Pre-checks: Verify file exists before attempting to read
58 #
59 # Best Practices Used:
60 # ✓ Specific exception catching (FileNotFoundError, PermissionError, etc.)
61 # ✓ Pre-validation checks (os.path.exists, os.path.isfile)
62 # ✓ Informative error messages
63 # ✓ Graceful error recovery
64 # ✓ Documentation with docstrings
65 # =====
66
67 # =====
68 # TEST CASES - 3 DIFFERENT SCENARIOS
69 # =====
70
71 print("=" * 70)
72 print("TASK #3: FILE HANDLING WITH EXCEPTION MANAGEMENT")
73 print("=" * 70)
74
75 # TEST 1: Non-existent file
76 print("\n--- TEST 1: Non-existent File ---")
77 result1 = read_file("nonexistent.txt")
78 assert "ERROR" in result1, "Test 1 failed: Should return error message"
79 assert "does not exist" in result1, "Test 1 failed: Should mention file doesn't exist"
80 print(f"Result: {result1}")
81 print("✓ Test 1 PASSED: Proper error handling for missing file")
```

Task_3_File_Not_Found_Bug.py > ...

```
75 # TEST 1: Non-existent file
76 print("\n--- TEST 1: Non-existent File ---")
77 result1 = read_file("nonexistent.txt")
78 assert "ERROR" in result1, "Test 1 failed: Should return error message"
79 assert "does not exist" in result1, "Test 1 failed: Should mention file doesn't exist"
80 print(f"Result: {result1}")
81 print("✓ Test 1 PASSED: Proper error handling for missing file")
82
83 # TEST 2: Create a temporary test file that exists
84 print("\n--- TEST 2: Existing File ---")
85 test_file = "test_sample.txt"
86 with open(test_file, 'w') as f:
87     f.write("This is a test file for reading.")
88
89 result2 = read_file(test_file)
90 assert "SUCCESS" in result2, "Test 2 failed: Should return success message"
91 assert "characters" in result2, "Test 2 failed: Should show character count"
92 print(f"Result: {result2}")
93 print("✓ Test 2 PASSED: Successfully reads existing file")
94
95 # Clean up test file
96 os.remove(test_file)
97
98 # TEST 3: Invalid path (directory instead of file)
99 print("\n--- TEST 3: Invalid Path (Directory) ---")
100 # Create a test directory
101 test_dir = "test_directory"
102 os.makedirs(test_dir, exist_ok=True)
103
```

```
assert "characters" in result2, "Test 2 failed: Should show character count"
print(f"Result: {result2}")
print("✓ Test 2 PASSED: Successfully reads existing file")

# Clean up test file
os.remove(test_file)

# TEST 3: Invalid path (directory instead of file)
print("\n--- TEST 3: Invalid Path (Directory) ---")
# Create a test directory
test_dir = "test_directory"
os.makedirs(test_dir, exist_ok=True)

result3 = read_file(test_dir)
assert "ERROR" in result3 or "not a valid file" in result3, "Test 3 failed: Should handle directory"
print(f"Result: {result3}")
print("✓ Test 3 PASSED: Proper handling of directory path")

# Clean up test directory
os.rmdir(test_dir)

print("\n" + "=" * 70)
print("ALL TESTS PASSED SUCCESSFULLY!")
print("=" * 70)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode> & C:\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant .py"

=====
TASK #3: FILE HANDLING WITH EXCEPTION MANAGEMENT
=====

--- TEST 1: Non-existent File ---
Result: ERROR: File 'nonexistent.txt' does not exist.
✓Test 1 PASSED: Proper error handling for missing file

--- TEST 2: Existing File ---
Result: SUCCESS: File content read (32 characters)
✓Test 2 PASSED: Successfully reads existing file

--- TEST 3: Invalid Path (Directory) ---
Result: ERROR: 'test_directory' is not a valid file.
✓Test 3 PASSED: Proper handling of directory path

=====
ALL TESTS PASSED SUCCESSFULLY!
=====

○ PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode>
```

Task Description-4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

```
class Car:  
    def start(self):  
        return "Car started"  
  
my_car = Car()  
print(my_car.drive()) # drive() is not defined
```

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.

Task_4_Undefined_Method_Bug.py > ...

```
1  # CORRECTED CODE
2  # Bug Fixed: Added the missing drive() method to the Car class
3  class Car:
4      """A simple Car class with basic vehicle operations"""
5
6      def __init__(self, model="Generic Car"):
7          """Initialize car with a model name"""
8          self.model = model
9          self.is_running = False
10         self.distance_traveled = 0
11
12         def start(self):
13             """Start the car engine"""
14             if self.is_running:
15                 return f"{self.model} is already running"
16             self.is_running = True
17             return f"{self.model} started successfully"
18
19         def drive(self, distance=10):
20             """
21             Drive the car for a specified distance.
22
23             Args:
24                 distance (int): Distance to drive in kilometers (default: 10)
25
26             Returns:
27                 str: Status message about the drive
28             """
29             if not self.is_running:
```

Task_4_Undefined_Method_Bug.py > ...

```
3  class Car:
19     def drive(self, distance=10):
20         return f"Cannot drive: {self.model} is not running. Call start() first."
31
32         self.distance_traveled += distance
33         return f"{self.model} drove {distance}km. Total distance: {self.distance_traveled}km"
34
35     def stop(self):
36         """Stop the car engine"""
37         if not self.is_running:
38             return f"{self.model} is already stopped"
39         self.is_running = False
40         return f"{self.model} stopped successfully"
41
42     def get_distance(self):
43         """Get total distance traveled"""
44         return self.distance_traveled
45
46     # =====
47     # EXPLANATION OF THE BUG:
48     # =====
49     # Original Code Problem:
50     #     class Car:
51     #         def start(self):
52     #             return "Car started"
53     #
54     #     my_car = Car()
55     #     print(my_car.drive()) # ← Calling drive() which doesn't exist!
56     #
```

Task_4_Undefined_Method_Bug.py > ...

```
53 #
54 #     my_car = Car()
55 #     print(my_car.drive()) # ← Calling drive() which doesn't exist!
56 #
57 # Error Encountered:
58 #     AttributeError: 'Car' object has no attribute 'drive'
59 #
60 # Root Cause:
61 #     The Car class has a start() method defined, but when the code tries
62 #     to call drive(), Python looks for that method in the class.
63 #     Since drive() doesn't exist, it raises an AttributeError.
64 #
65 # Why This Is a Bug:
66 #     1. The method is not defined in the class
67 #     2. Python can't find the attribute/method when called
68 #     3. Program crashes with AttributeError
69 #     4. No graceful error handling
70 #
71 # How to Debug:
72 #     Step 1: Read the error message carefully
73 #             "Car object has no attribute 'drive'"
74 #             This tells us exactly what's wrong
75 #
76 #     Step 2: Check if the method exists in the class
77 #             Look at class definition → drive() is missing
78 #
79 #     Step 3: Decide on solution:
80 #             Option A: Add the missing method ✓ (recommended)
81 #             Option B: Change method call to use existing method
```

```

Task_4_Undefined_Method_Bug.py > ...
81  #           Option B: Change method call to use existing method
82  #
83  #       Step 4: Implement the solution
84  #           Add drive() method with proper implementation
85  #
86  # Solution Applied:
87  #       ✓ Added drive() method to the Car class
88  #       ✓ Proper method signature with distance parameter
89  #       ✓ Logic to check if car is running before driving
90  #       ✓ Tracking total distance traveled
91  #       ✓ User-friendly return messages
92  #       ✓ Added docstrings for documentation
93  #       ✓ Added __init__() for initialization
94  #       ✓ Added other helper methods (stop, get_distance)
95  # =====
96
97  # =====
98  # TEST CASES - 3 SCENARIOS
99  # =====
100
101  print("=" * 70)
102  print("TASK #4: CLASS METHOD DEBUGGING - UNDEFINED METHOD")
103  print("=" * 70)
104
105  # Initialize a car object
106  my_car = Car(model="Tesla Model 3")
107
108  # TEST 1: Start the car and verify start() method works
109  print("\n--- TEST 1: Start Method Works ---")
110  start_result = my_car.start()

```

Task_4_Undefined_Method_Bug.py > ...

```
110 start_result = my_car.start()
111 assert "started successfully" in start_result, "Test 1 failed: Start should succeed"
112 assert my_car.is_running == True, "Test 1 failed: Car should be running"
113 print(f"Result: {start_result}")
114 print("✓ Test 1 PASSED: start() method works correctly")
115
116 # TEST 2: Drive the car and verify drive() method works (the previously undefined method)
117 print("\n--- TEST 2: Drive Method Works (Previously Undefined) ---")
118 drive_result = my_car.drive(distance=50)
119 assert "drove 50km" in drive_result, "Test 2 failed: Drive should record distance"
120 assert my_car.distance_traveled == 50, "Test 2 failed: Distance should be tracked"
121 assert "Total distance: 50km" in drive_result, "Test 2 failed: Should show total distance"
122 print(f"Result: {drive_result}")
123 print("✓ Test 2 PASSED: drive() method works correctly (BUG FIXED!)")
124
125 # TEST 3: Stop the car and verify error handling (trying to drive when stopped)
126 print("\n--- TEST 3: Error Handling - Drive When Not Running ---")
127 stop_result = my_car.stop()
128 assert "stopped successfully" in stop_result, "Test 3 failed: Stop should succeed"
129 assert my_car.is_running == False, "Test 3 failed: Car should not be running"
130
131 # Try to drive when car is stopped - should return error message, not crash
132 drive_error = my_car.drive(distance=20)
133 assert "Cannot drive" in drive_error or "not running" in drive_error, \
134     "Test 3 failed: Should return error when driving stopped car"
135 assert my_car.distance_traveled == 50, \
136     "Test 3 failed: Distance should not increase when not running"
137 print(f"Result: {drive_error}")
138 print("✓ Test 3 PASSED: Error handling works - no crash when method called on stopped car")
```

```
# Try to drive when car is stopped - should return error message, not crash
drive_error = my_car.drive(distance=20)
assert "Cannot drive" in drive_error or "not running" in drive_error, \
    "Test 3 failed: Should return error when driving stopped car"
assert my_car.distance_traveled == 50, \
    "Test 3 failed: Distance should not increase when not running"
print(f"Result: {drive_error}")
print("✓ Test 3 PASSED: Error handling works - no crash when method called on stopped car")

print("\n" + "=" * 70)
print("ALL TESTS PASSED SUCCESSFULLY!")
print("=" * 70)

# Additional validation
print("\n--- Additional Validation ---")
print(f"Total distance traveled: {my_car.get_distance()}km")
print(f"Car model: {my_car.model}")
print(f"Car is running: {my_car.is_running}")
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode> cd "c:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode"; python "Task_4_undefined_method_bug.py"
=====
TASK #4: CLASS METHOD DEBUGGING - UNDEFINED METHOD
=====

--- TEST 1: Start Method Works ---
Result: Tesla Model 3 started successfully
✓ Test 1 PASSED: start() method works correctly

--- TEST 2: Drive Method Works (Previously Undefined) ---
Result: Tesla Model 3 drove 50km. Total distance: 50km
✓ Test 2 PASSED: drive() method works correctly (BUG FIXED!)

--- TEST 3: Error Handling - Drive When Not Running ---
Result: Cannot drive: Tesla Model 3 is not running. Call start() first.
✓ Test 3 PASSED: Error handling works - no crash when method called on stopped car

=====
ALL TESTS PASSED SUCCESSFULLY!
=====

--- Additional Validation ---
Total distance traveled: 50km
Car model: Tesla Model 3
Car is running: False
● PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode>
```

Task Description - 5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```
def add_five(value):
```

```
    return value + 5
```

```
print(add_five("10"))
```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

```

# CORRECTED CODE - TWO SOLUTIONS FOR TypeError
# Bug Fixed: TypeError due to mixing string and integer

# =====
# SOLUTION 1: TYPE CASTING (CONVERTING STRING TO INTEGER)
# =====
# Best when: You need to perform mathematical operations
# Purpose: Convert the string value to an integer before performing addition
# Result: Returns an integer value

def add_five_solution1(value):
    """
    Solution 1: Type Casting
    Converts the input to an integer and adds 5 to it.

    Args:
        value: Can be a string, integer, or float

    Returns:
        int: The result of adding 5 to the converted integer value

    Raises:
        ValueError: If value cannot be converted to an integer
    """
    try:
        # Convert value to integer first, then add 5
        return int(value) + 5
    except ValueError:
        return f"ERROR: Cannot convert '{value}' to an integer"

```

```

30
31
32 # =====
33 # SOLUTION 2: STRING CONCATENATION (TREATING BOTH AS STRINGS)
34 # =====
35 # Best when: You want to combine values as strings
36 # Purpose: Convert the integer to a string and concatenate with string value
37 # Result: Returns a string value
38
39 def add_five_solution2(value):
40     """
41     Solution 2: String Concatenation
42     Converts the integer 5 to a string and concatenates it with the input.
43
44     Args:
45         value: Any value that can be converted to a string
46
47     Returns:
48         str: The concatenated string of value and "5"
49     """
50     # Convert both to strings and concatenate
51     return str(value) + "5"
52
53
54 # =====
55 # EXPLANATION OF THE BUG:
56 # =====
57 # Original Code Problem:
58 #     def add_five(value):

```

```
#  
# print(add_five("10"))  
#  
# Error Encountered:  
#     TypeError: can only concatenate str (not "int") to str  
#  
# Root Cause:  
#     Python has strict type rules:  
#     - String "10" and Integer 5 cannot be directly added  
#     - You cannot mix data types in operations without explicit conversion  
#     - Python doesn't know how to combine string + integer  
#  
# Type System in Python:  
#     "10" → String type (text)  
#     10 → Integer type (number)  
#     10.0 → Float type (decimal number)  
#  
#     ✗ "10" + 5 (TypeError: can't mix string and int)  
#     ✓ int("10") + 5 (Valid: converts "10" to 10, then adds)  
#     ✓ "10" + "5" (Valid: concatenates to "105")  
#  
# Why This Happens:  
#     Python is a STRONGLY TYPED language  
#     It enforces type compatibility in operations  
#     String concatenation (+) ≠ Integer addition (+)  
#     Python requires explicit type conversion  
#  
# TWO VALID SOLUTIONS:  
#  
#     SOLUTION 1 - Type Casting (Math Operation):
```

```

89 # SOLUTION 1 - Type Casting (Math Operation):
90 #     int(value) + 5
91 #     "10" → 10 → 10 + 5 = 15 (integer)
92 #     Use when: You need mathematical results
93 #
94 # SOLUTION 2 - String Concatenation:
95 #     str(value) + "5"
96 #     10 → "10" → "10" + "5" = "105" (string)
97 #     Use when: You need to combine strings
98 #
99 # Key Differences:
100 #     Casting:     "10" + 5 = 15 (math operation)
101 #     Concatenation: "10" + 5 = "105" (string combination)
102 # =====
103
104 # =====
105 # COMPREHENSIVE TEST CASES - 3 SCENARIOS EACH
106 # =====
107
108 print("=" * 80)
109 print("TASK #5: TypeError DEBUGGING LAB - MIXING STRINGS AND INTEGERS")
110 print("=" * 80)
111
112 print("\n" + "=" * 80)
113 print("SOLUTION 1: TYPE CASTING (Convert string to integer)")
114 print("=" * 80)
115
116 # TEST 1a: String input that can be converted to integer
117 print("\n--- TEST 1a: String Integer Input ---")
118 result1a = add_five_solution1("10")

```

Task_5_TypeError_Bug.py > ...

```
118 result1a = add_five_solution1("10")
119 assert result1a == 15, f"Test 1a failed: Expected 15, got {result1a}"
120 assert isinstance(result1a, int), "Test 1a failed: Result should be integer type"
121 print(f"Input: add_five_solution1('10')")
122 print(f"Result: {result1a}")
123 print(f"Type: {type(result1a).__name__}")
124 print("✓ Test 1a PASSED: String '10' converted to int, added 5, result = 15")
125
126 # TEST 1b: Actual integer input
127 print("\n--- TEST 1b: Actual Integer Input ---")
128 result1b = add_five_solution1(7)
129 assert result1b == 12, f"Test 1b failed: Expected 12, got {result1b}"
130 assert isinstance(result1b, int), "Test 1b failed: Result should be integer type"
131 print(f"Input: add_five_solution1(7)")
132 print(f"Result: {result1b}")
133 print(f"Type: {type(result1b).__name__}")
134 print("✓ Test 1b PASSED: Integer 7 added with 5, result = 12")
135
136 # TEST 1c: Negative integer input
137 print("\n--- TEST 1c: Negative Integer Input (Edge Case) ---")
138 result1c = add_five_solution1("-3")
139 assert result1c == 2, f"Test 1c failed: Expected 2, got {result1c}"
140 assert isinstance(result1c, int), "Test 1c failed: Result should be integer type"
141 print(f"Input: add_five_solution1('-3')")
142 print(f"Result: {result1c}")
143 print(f"Type: {type(result1c).__name__}")
144 print("✓ Test 1c PASSED: Negative integer converted correctly, -3 + 5 = 2")
145
146 print("\n" + "=" * 80)
```

```

Task_5_TypeError_Bug.py > ...
147 print( SOLUTION 2: STRING CONCATENATION (treat both as strings) )
148 print("=" * 80)
149
150 # TEST 2a: String input concatenated with "5"
151 print("\n--- TEST 2a: String Concatenation ---")
152 result2a = add_five_solution2("10")
153 assert result2a == "105", f"Test 2a failed: Expected '105', got {result2a}"
154 assert isinstance(result2a, str), "Test 2a failed: Result should be string type"
155 print(f"Input: add_five_solution2('10')")
156 print(f"Result: {result2a}")
157 print(f"Type: {type(result2a).__name__}")
158 print("✓ Test 2a PASSED: String '10' concatenated with '5', result = '105'")
159
160 # TEST 2b: Integer input converted to string, then concatenated
161 print("\n--- TEST 2b: Integer Input Converted to String ---")
162 result2b = add_five_solution2(7)
163 assert result2b == "75", f"Test 2b failed: Expected '75', got {result2b}"
164 assert isinstance(result2b, str), "Test 2b failed: Result should be string type"
165 print(f"Input: add_five_solution2(7)")
166 print(f"Result: {result2b}")
167 print(f"Type: {type(result2b).__name__}")
168 print("✓ Test 2b PASSED: Integer 7 converted to string '7', concatenated with '5'")
169
170 # TEST 2c: Mixed type input (demonstrates flexibility)
171 print("\n--- TEST 2c: Mixed Type Input ---")
172 result2c = add_five_solution2("ABC")
173 assert result2c == "ABC5", f"Test 2c failed: Expected 'ABC5', got {result2c}"
174 assert isinstance(result2c, str), "Test 2c failed: Result should be string type"
175 print(f"Input: add_five_solution2('ABC')")
176 print(f"Result: {result2c}")

```


Task_5_TypeError_Bug.py > ...

```
176 print(f"Result: {result2c}")
177 print(f"Type: {type(result2c).__name__}")
178 print("✓ Test 2c PASSED: String 'ABC' concatenated with '5', result = 'ABC5'")
179
180 print("\n" + "=" * 80)
181 print("COMPARISON TABLE")
182 print("=" * 80)
183 print("\n{:<20} {:<25} {:<25} {:<20}".format(
184     "Input", "Solution 1 (Casting)", "Solution 2 (Concat)", "Use When"
185 ))
186 print("-" * 90)
187 print("{:<20} {:<25} {:<25} {:<20}".format(
188     "10", "15 (integer)", "'105' (string)", "Math operation"
189 ))
190 print("{:<20} {:<25} {:<25} {:<20}".format(
191     "7", "12 (integer)", "'75' (string)", "Data combination"
192 ))
193 print("{:<20} {:<25} {:<25} {:<20}".format(
194     "20", "25 (integer)", "'205' (string)", "Flexibility needed"
195 ))
196
197 print("\n" + "=" * 80)
198 print("ALL TESTS PASSED SUCCESSFULLY!")
199 print("=" * 80)
200
201 print("\n" + "=" * 80)
202 print("RECOMMENDED USAGE")
203 print("=" * 80)
204 print("""
```

```

190     ))
191     print("{:<20} {:<25} {:<25} {:<20}".format(
192         '7', "12 (integer)", '"75" (string)', "Data combination"
193     ))
194     print("{:<20} {:<25} {:<25} {:<20}".format(
195         '"20"', "25 (integer)", '"205" (string)', "Flexibility needed"
196     ))
197
198     print("\n" + "=" * 80)
199     print("ALL TESTS PASSED SUCCESSFULLY!")
200     print("=" * 80)
201
202     print("\n" + "=" * 80)
203     print("RECOMMENDED USAGE")
204     print("=" * 80)
205     print("""
206     Use SOLUTION 1 (Type Casting) when:
207         ✓ You need to perform mathematical operations
208         ✓ The final result should be a number
209         ✓ Input might be string representation of numbers
210         ✓ You want: "10" + 5 = 15
211
212     Use SOLUTION 2 (String Concatenation) when:
213         ✓ You need to combine values as text
214         ✓ The final result should be a string
215         ✓ You're building formatted output
216         ✓ You want: "10" + 5 = "105"
217     """)
218

```

```
PS C:\Users\rishi\OneDrive\Desktop\UNI\SEM - VI\AI Assistant Coding\vscode> & C:\Users\rishi\AppData\Local\Microsoft\WindowsApps\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/vscode/"
COMPARISON TABLE
```

Input	Solution 1 (Casting)	Solution 2 (Concat)	Use When
"10"	15 (integer)	"105" (string)	Math operation
7	12 (integer)	"75" (string)	Data combination
"20"	25 (integer)	"205" (string)	Flexibility needed

ALL TESTS PASSED SUCCESSFULLY!

RECOMMENDED USAGE

Use SOLUTION 1 (Type Casting) when:

- ✓ You need to perform mathematical operations
- ✓ The final result should be a number
- ✓ Input might be string representation of numbers
- ✓ You want: "10" + 5 = 15
- ✓ You want: "10" + 5 = 15
- ✓ You want: "10" + 5 = 15

Use SOLUTION 2 (String Concatenation) when:

- ✓ You need to combine values as text
- ✓ The final result should be a string
- ✓ You're building formatted output
- ✓ You want: "10" + 5 = "105"